

NGHIÊN CỨU XÂY DỰNG HỆ THỐNG THU THẬP DỮ LIỆU SONG SONG SỬ DỤNG CƠ CHẾ TRUY CẬP BỘ NHỚ TRỰC TIẾP VÀ HỆ ĐIỀU HÀNH THỜI GIAN THỰC TRÊN VI ĐIỀU KHIỂN STM32

Lê Tiến Đạt¹, Nguyễn Hoàng Khải² và Trương Phong Tuyên³

¹ Ngành Kỹ thuật Máy tính K49, Khoa Điện tử - Viễn thông, Trường Bách khoa, Đại học Cần Thơ

² Ngành Kỹ thuật Máy tính K49, Khoa Điện tử - Viễn thông, Trường Bách khoa, Đại học Cần Thơ

³ Khoa Điện tử - Viễn thông, Trường Bách khoa, Đại học Cần Thơ

Thông tin chung:

Ngày nhận bài:

Ngày nhận bài sửa:

Ngày duyệt đăng:

Title:

Research and Development of a Parallel Data Acquisition System Using Direct Memory Access and RT-Thread Operating System on STM32 Microcontrollers

Từ khóa:

Truy cập bộ nhớ trực tiếp, Hệ điều hành thời gian thực (RTOS), thu thập dữ liệu song song, STM32

Keywords:

Direct Memory Access, Real-Time Operating System (RTOS), parallel data acquisition, STM32

ABSTRACT

This study presents the design and implementation of a high-performance parallel data acquisition system based on the STM32 microcontroller, aiming to overcome the limitations of sequential data processing in multi-sensor environments. In conventional approaches, the CPU must directly handle communication tasks, leading to increased latency and inefficient resource utilization as the number of sensors increases. The proposed system employs two ADS1115 ADC modules operating in parallel under STM32 control. By integrating Direct Memory Access (DMA) with the RT-Thread real-time operating system, the I2C communication process is fully automated, allowing data transfer to be handled by hardware instead of the CPU, reducing processor workload and improving efficiency. Additionally, RT-Thread enables efficient task scheduling and concurrent execution, allowing data acquisition and processing to occur simultaneously. Experimental results show stable operation with continuous and synchronized data collection across multiple channels. The proposed approach provides an effective solution for optimizing CPU performance in real-time analog signal monitoring applications.

TÓM TẮT

Nghiên cứu này trình bày việc thiết kế và xây dựng hệ thống thu thập dữ liệu song song hiệu năng cao sử dụng vi điều khiển STM32, nhằm khắc phục hạn chế của phương pháp xử lý tuần tự trong các hệ thống đa cảm biến. Trong các phương pháp truyền thống, CPU phải trực tiếp tham gia vào quá trình giao tiếp dữ liệu, gây ra độ trễ và làm giảm hiệu suất khi số lượng cảm biến tăng. Giải pháp đề xuất sử dụng hai module ADC ADS1115 hoạt động song song dưới sự điều khiển của STM32. Bằng cách tích hợp cơ chế DMA và hệ điều hành thời gian thực RT-Thread, quá trình truyền dữ liệu qua I2C được tự động hóa, giúp giảm tải cho CPU và nâng cao hiệu quả sử dụng tài nguyên. RT-Thread cho phép quản lý đa luồng hiệu quả, giúp hệ thống có thể thực hiện đồng thời việc thu thập và xử lý dữ liệu mà không gây nghẽn. Kết quả thực nghiệm cho thấy hệ thống hoạt động ổn định, dữ liệu được thu thập liên tục và đồng bộ giữa các module. Giải pháp này cho thấy hiệu quả trong việc tối ưu hóa hiệu suất CPU, phù hợp với các ứng dụng giám sát tín hiệu tương tự theo thời gian thực.

1. GIỚI THIỆU

1.1. Bối cảnh và tính cấp thiết của đề tài.

Trong các hệ thống giám sát công nghiệp, việc thu thập dữ liệu từ nhiều cảm biến đòi hỏi tốc độ cao và tính liên tục để đảm bảo khả năng phản hồi kịp thời. Khi số lượng cảm biến tăng, yêu cầu về hiệu suất xử lý của hệ thống cũng trở nên cao hơn.

Tuy nhiên, các phương pháp truyền thống như hỏi vòng (polling) hoặc ngắt (interrupt) buộc CPU phải trực tiếp tham gia vào quá trình giao tiếp dữ liệu qua I2C, gây ra các khoảng chờ (wait-states) và làm giảm hiệu quả sử dụng tài nguyên.

Do đó, việc tìm kiếm giải pháp giúp giảm tải cho CPU và nâng cao hiệu suất thu thập dữ liệu là cần thiết trong các hệ thống nhúng hiện đại.

1.2. Xác định vấn đề và tổng quan tài liệu

Khi sử dụng nhiều module ADC ADS1115 trong cùng một hệ thống, một thách thức chính là tốc độ giao tiếp I2C thấp hơn so với tốc độ xử lý của vi điều khiển STM32 [1]. Nếu áp dụng phương pháp lập trình tuần tự, CPU phải chờ hoàn tất quá trình truyền nhận dữ liệu trước khi thực hiện các tác vụ khác, làm giảm hiệu suất hệ thống [2].

Do đó, vấn đề đặt ra là cần xây dựng một phương pháp điều khiển hiệu quả hơn nhằm giảm thời gian chờ và tối ưu hóa việc sử dụng tài nguyên CPU. Một hướng tiếp cận phù hợp là kết hợp cơ chế truy cập bộ nhớ trực tiếp (DMA) với hệ điều hành thời gian thực RT-Thread [3], cho phép tự động hóa quá trình truyền dữ liệu và hỗ trợ xử lý song song.

1.3. Giải pháp đề xuất

Nghiên cứu đề xuất giải pháp tích hợp cơ chế truy cập bộ nhớ trực tiếp (DMA) trên vi điều khiển STM32 kết hợp với hệ điều hành thời gian thực RT-Thread. Trong đó, DMA đảm nhiệm việc truyền dữ liệu giữa các ngoại vi và bộ nhớ mà không cần CPU can thiệp trực tiếp.

Đồng thời, RT-Thread được sử dụng để quản lý và điều phối các tác vụ theo cơ chế đa luồng, cho phép hệ thống thực hiện đồng thời việc thu thập và xử lý dữ liệu. Nhờ đó, hiệu suất

hoạt động được cải thiện và tải xử lý của CPU được giảm đáng kể.

1.4. Tính mới của nghiên cứu

Điểm mới của nghiên cứu là việc thay thế phương pháp xử lý tuần tự bằng cơ chế thu thập dữ liệu song song tự động. Thông qua việc kết hợp DMA và RT-Thread, hệ thống phân tách rõ ràng giữa quá trình thu thập dữ liệu (phần cứng đảm nhiệm) và quá trình xử lý dữ liệu (phần mềm đảm nhiệm). Nhờ đó, các module ADS1115 có thể hoạt động đồng thời mà không làm gián đoạn CPU, góp phần nâng cao hiệu suất và khả năng mở rộng của hệ thống.

1.5. Mục tiêu nghiên cứu

Mục tiêu của nghiên cứu là nâng cao hiệu suất thu thập dữ liệu trong hệ thống nhúng sử dụng nhiều cảm biến. Cụ thể, nghiên cứu tập trung triển khai cơ chế truy cập bộ nhớ trực tiếp (DMA) cho giao tiếp I2C trên vi điều khiển STM32 nhằm giảm thời gian chờ của CPU.

Bên cạnh đó, hệ điều hành thời gian thực RT-Thread được sử dụng để tổ chức và xử lý song song dữ liệu từ các module ADS1115. Qua đó, hệ thống hướng đến việc tối ưu hóa tài nguyên xử lý và cải thiện khả năng đáp ứng trong các ứng dụng giám sát tín hiệu theo thời gian thực.

2. PHƯƠNG PHÁP NGHIÊN CỨU

2.1. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là hệ thống thu thập dữ liệu sử dụng vi điều khiển STM32F411VET6 làm khối xử lý trung tâm. Vi điều khiển này được lựa chọn nhờ khả năng xử lý cao, hỗ trợ DMA và phù hợp cho các ứng dụng đa nhiệm.

Hệ thống sử dụng hai module ADC ADS1115 giao tiếp qua I2C để thu thập tín hiệu từ các cảm biến. Các cảm biến được sử dụng bao gồm cảm biến nhiệt độ LM35 và cảm biến ánh sáng LDR nhằm mô phỏng các tín hiệu tương tự trong môi trường thực tế.

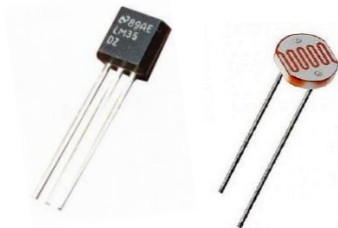
Toàn bộ hệ thống được vận hành trên nền tảng hệ điều hành thời gian thực RT-Thread, kết hợp với cơ chế DMA để hỗ trợ truyền dữ liệu song song và giảm tải cho CPU.



Hình 1. Vi điều khiển STM32F411VET6



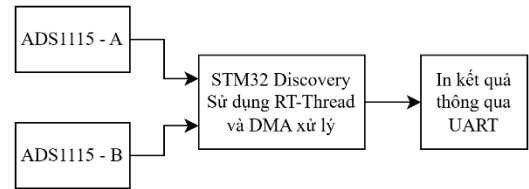
Hình 2. Module ADC 16-bit ADS1115



Hình 3. Cảm biến LM35 và LDR

2.2. Tổng quan về sơ đồ khối hệ thống

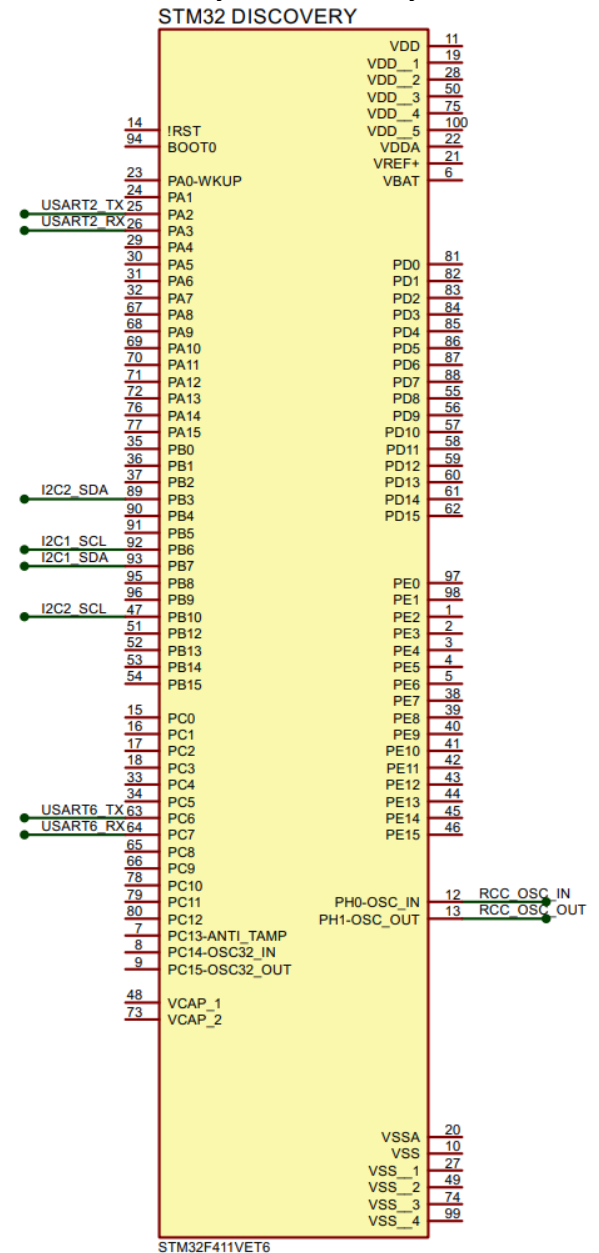
Sơ đồ khối hệ thống thể hiện vi điều khiển STM32 là trung tâm, nhận dữ liệu từ hai module ADC ADS1115 qua giao tiếp I2C. DMA hỗ trợ truyền dữ liệu tự động, giảm tải cho CPU, trong khi RT-Thread quản lý các tác vụ, giúp hệ thống hoạt động song song và ổn định. Được trình bày tại Hình 4.



Hình 4. Sơ đồ khối của hệ thống

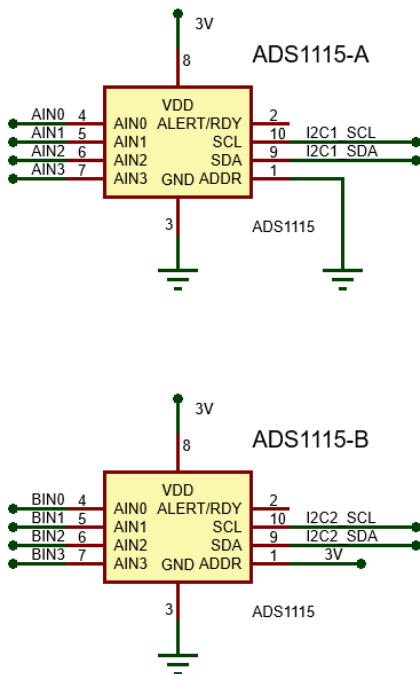
2.3. Thiết kế hệ thống

Sơ đồ nguyên lý kết nối của vi điều khiển STM32 Discovery. Được trình bày tại Hình 5.



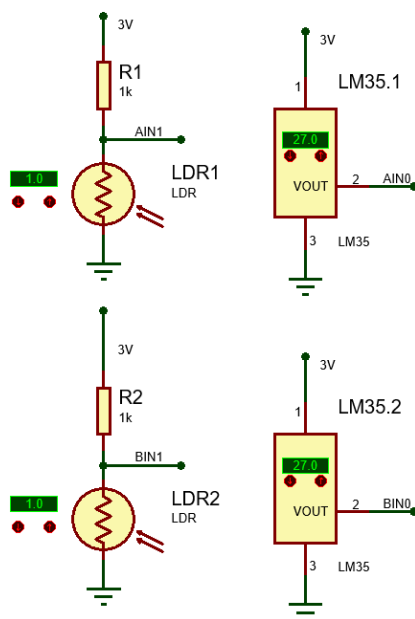
Hình 5. Sơ đồ nguyên lý của vi xử lý STM32

Sơ đồ nguyên lý kết nối phần cứng của khối module ADS1115 được trình bày tại Hình 6.



Hình 6. Sơ đồ nguyên lý kết nối phần cứng của khối module ADS1115

Sơ đồ nguyên lý kết nối phần cứng của cảm biến được trình bày tại Hình 7.



Hình 7. Sơ đồ nguyên lý kết nối phần cứng của cảm biến

2.4. Công cụ nghiên cứu và kỹ thuật thu thập thông tin

Trong hệ thống đề xuất, cơ chế truyền tải dữ liệu được thực hiện thông qua bộ điều khiển truy cập bộ nhớ trực tiếp (DMA), cho phép dữ liệu được chuyển trực tiếp từ các ngoại vi vào bộ nhớ RAM mà không cần sự can thiệp liên tục của CPU. Nhờ đó, CPU không phải chờ đợi trong quá trình truyền dữ liệu, giúp giảm tải đáng kể và nâng cao hiệu suất xử lý tổng thể của hệ thống.

Quá trình phát triển và lập trình được thực hiện trên phần mềm STM32CubeIDE do STMicroelectronics cung cấp. Công cụ này tích hợp sẵn STM32CubeMX, hỗ trợ cấu hình phần cứng, thiết lập các ngoại vi như I2C, USART, DMA, đồng thời cho phép viết mã nguồn C/C++, biên dịch và nạp chương trình xuống vi điều khiển một cách thuận tiện và đồng bộ.

Việc thiết kế và kiểm tra sơ đồ nguyên lý được thực hiện bằng phần mềm Proteus 8. Công cụ này hỗ trợ mô phỏng hoạt động của mạch, giúp đánh giá tính đúng đắn của kết nối phần cứng trước khi triển khai trên hệ thống thực tế, từ đó giảm thiểu sai sót trong quá trình phát triển.

Để phục vụ cho việc giám sát và thu thập dữ liệu, phần mềm Tera Term được sử dụng như một công cụ giả lập thiết bị đầu cuối (Terminal Emulator). Phần mềm này được cấu hình giao tiếp với vi điều khiển STM32 thông qua cổng USART với tốc độ baudrate 115200, cho phép hiển thị, theo dõi và ghi nhận dữ liệu cảm biến theo thời gian thực một cách trực quan.

Về kỹ thuật triển khai, hệ điều hành thời gian thực RT-Thread được sử dụng để tổ chức các tác vụ dưới dạng các luồng (thread) độc lập. Hệ thống áp dụng cơ chế phân chia thời gian (time-slicing) để điều phối CPU giữa các luồng, đảm bảo khả năng xử lý đa nhiệm hiệu quả và tránh tình trạng nghẽn hệ thống. Đồng thời, DMA được kích hoạt để tự động truyền dữ liệu từ các module ADS1115 vào vùng nhớ đệm, giúp quá trình thu thập dữ liệu diễn ra liên tục, ổn định và không làm gián đoạn CPU.

Dữ liệu sau khi được xử lý sẽ được gửi định kỳ đến Tera Term thông qua giao tiếp USART, phục vụ cho việc quan sát, lưu trữ và phân tích kết quả. Điều này giúp người dùng dễ dàng

đánh giá hoạt động của hệ thống cũng như chất lượng dữ liệu thu thập được.

2.5. Quy trình thử nghiệm

Quy trình thử nghiệm được thực hiện bằng cách thu thập dữ liệu từ môi trường theo thời gian thực trong một khoảng thời gian xác định, nhằm đánh giá tính ổn định và khả năng hoạt động liên tục của hệ thống. Các tín hiệu từ cảm biến được lấy mẫu nhiều lần để đảm bảo dữ liệu thu thập có tính đại diện và phản ánh đúng trạng thái của môi trường.

Trong quá trình thử nghiệm, hệ thống sử dụng phương pháp kết hợp giữa cơ chế truy cập bộ nhớ trực tiếp (DMA) và hệ điều hành thời gian thực RT-Thread để thực hiện thu thập dữ liệu. DMA đảm nhiệm việc truyền dữ liệu từ các module ADC vào bộ nhớ một cách tự động, trong khi RT-Thread điều phối các tác vụ theo cơ chế đa luồng, giúp hệ thống hoạt động hiệu quả hơn.

Nhờ sự phối hợp này, dữ liệu từ các cảm biến được lấy mẫu, truyền và xử lý gần như đồng thời mà không làm gián đoạn CPU. Điều này giúp hệ thống duy trì hoạt động ổn định, đảm bảo khả năng thu thập dữ liệu liên tục và phản hồi kịp thời trong suốt quá trình thử nghiệm.

2.6. Nguyên lý hoạt động của chương trình

2.6.1. Giai đoạn khởi tạo

Khi hệ thống được kích hoạt, chương trình chính tiến hành cấu hình phần cứng bao gồm xung nhịp hệ thống và các giao tiếp như I2C, USART. Sau đó, hệ điều hành thời gian thực RT-Thread được khởi động để quản lý toàn bộ hoạt động của hệ thống.

Hai Semaphore nhị phân (`ads1_sem` và `ads2_sem`) được khởi tạo với giá trị ban đầu bằng 0 nhằm đồng bộ giữa tác vụ phần mềm và quá trình truyền dữ liệu bằng DMA. Đồng thời, hai luồng độc lập `t1` và `t2` được tạo ra để quản lý hai module cảm biến riêng biệt, và được đưa vào trạng thái sẵn sàng.

Sau khi hoàn tất khởi tạo, RT-Thread bắt đầu điều phối việc thực thi các luồng dựa trên cơ chế ưu tiên và thuật toán chia thời gian (time-slicing), cho phép hệ thống vận hành theo mô hình đa nhiệm.

2.6.2. Chu trình hoạt động song song

Hai tác vụ `t1` và `t2` được thiết kế với cấu trúc hoạt động giống nhau, trong đó mỗi tác vụ đảm nhiệm việc giao tiếp và thu thập dữ liệu từ một module **ADS1115** riêng biệt. Nhờ cơ chế đa nhiệm của hệ điều hành, hai tác vụ này có thể hoạt động song song, giúp nâng cao hiệu suất thu thập dữ liệu của toàn hệ thống. Quá trình hoạt động của mỗi tác vụ được thực hiện theo các bước sau:

Bước 1: Tác vụ xác định kênh ADC cần đọc, sau đó gọi hàm `HAL_I2C_Master_Transmit_DMA` để gửi dữ liệu cấu hình đến module **ADS1115**. Khi hàm này được gọi, bộ điều khiển DMA được kích hoạt để thực hiện việc truyền dữ liệu qua giao tiếp I2C mà không cần CPU can thiệp trực tiếp.

Bước 2: Ngay sau khi lệnh DMA được thực thi, tác vụ phần mềm chủ động thực hiện lệnh `rt_sem_take`. Do Semaphore đang có giá trị 0, tác vụ này lập tức bị chuyển từ trạng thái đang chạy sang trạng thái chờ và tạm ngưng sử dụng CPU và cho phép hệ điều hành cấp quyền thực thi cho các tác vụ khác đang ở trạng thái sẵn sàng.

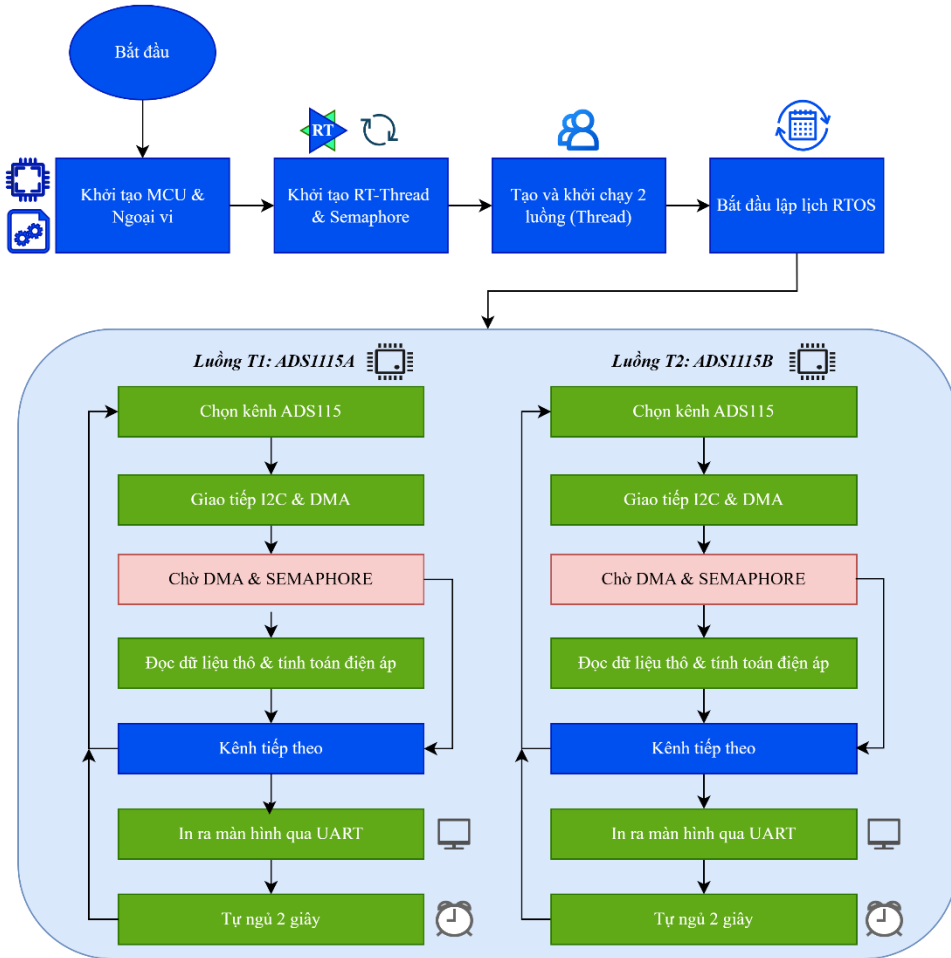
Bước 3: Trong khi tác vụ phần mềm đang bị chặn, bộ ngoại vi DMA và I2C tự thực hiện việc di chuyển dữ liệu theo đường I2C đến cảm biến (trong nền) mà không cần CPU can thiệp.

Bước 4: Khi DMA hoàn tất việc di chuyển dữ liệu, nó kích hoạt một **tín hiệu ngắt** hoàn tất đến CPU. Trình xử lý ngắt được kích hoạt, tạm dừng tác vụ hiện tại để thực thi hàm Callback tương ứng. Tại đây, hàm Callback thực hiện lệnh `rt_sem_release`, làm tăng giá trị Semaphore lên 1 và đánh thức tác vụ đang chờ ở Bước 2, chuyển nó trở lại trạng thái sẵn sàng.

Bước 5: Quy trình ghi config (Bước 1-4) được lặp lại tương tự cho lệnh đọc kết quả **HAL_I2C_Master_Receive_DMA**. Sau khi nhận được dữ liệu thô, tác vụ thức dậy, thực hiện ghép byte (2 byte nhận được thành số nguyên 16-bit), quy đổi sang điện áp và xuất kết quả ra màn hình.

2.7. Lưu đồ giải thuật

Lưu đồ giải thuật cấu chương trình được trình bày tại Hình 8.



Hình 8: Lưu đồ giải thuật

Lưu đồ mô tả quá trình hoạt động của hệ thống gồm hai giai đoạn: khởi tạo và xử lý song song.

Ban đầu, hệ thống khởi tạo MCU, các ngoại vi (I2C, USART, DMA), sau đó khởi động RT-Thread và tạo hai luồng t1, t2. Bộ lập lịch RTOS bắt đầu phân chia CPU cho các luồng.

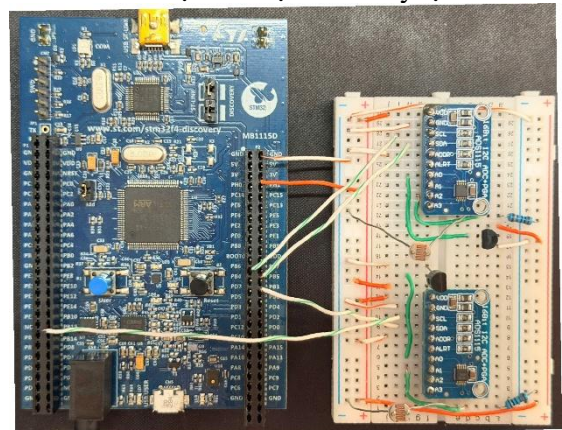
Trong quá trình hoạt động, mỗi luồng thực hiện: chọn kênh ADC → truyền dữ liệu qua I2C kết hợp DMA → chờ hoàn tất bằng Semaphore → đọc và xử lý dữ liệu → chuyển sang kênh tiếp theo. Kết quả được gửi ra màn hình qua USART.

Hai luồng hoạt động song song, giúp hệ thống thu thập dữ liệu liên tục và giảm tải cho CPU.

3. KẾT QUẢ VÀ THẢO LUẬN

3.1. Kết quả mô hình thực tế

Mô hình thực tế được trình bày tại Hình 9.



Hình 9. Mô hình thực tế của hệ thống

3.2. Kết quả thu thập dữ liệu

Kết quả thu thập dữ liệu được hiển thị trên phần mềm Tera Term thông qua hai cổng giao tiếp USART. Cụ thể, cổng **USART2** được sử dụng để xuất dữ liệu từ module **ADS1115A**, trong khi cổng **USART6** dùng để hiển thị dữ liệu từ module **ADS1115B**.

Đối với mỗi module, dữ liệu được hiển thị dưới hai dạng gồm giá trị thô và giá trị điện áp tương ứng. Trong đó, **ADS1 CH0** và **ADS1 CH1** biểu diễn giá trị thô đọc được từ các

kênh của module **ADS1115A**, còn **ADS1V CH0** và **ADS1V CH1** thể hiện giá trị điện áp đã được quy đổi.

Tương tự, **ADS2 CH0** và **ADS2 CH1** là giá trị thô từ module **ADS1115B**, trong khi **ADS2V CH0** và **ADS2V CH1** biểu diễn giá trị điện áp tương ứng sau khi xử lý.

Việc hiển thị đồng thời dữ liệu từ hai module cho thấy hệ thống có khả năng thu thập và xử lý dữ liệu song song một cách ổn định và liên tục.

Tick	Module	Channel	Raw Value	Converted Value
222049	ADS1	CH0	2.960250	0.328625
222049	ADS1	CH1	2.960000	0.328750
226159	ADS1	CH0	2.960125	0.328750
226159	ADS1	CH1	2.960125	0.328750
228214	ADS1	CH0	2.960125	0.328625
228214	ADS1	CH1	2.960125	0.328625
230269	ADS1	CH0	2.960000	0.328500
230269	ADS1	CH1	2.960000	0.328500
232324	ADS1	CH0	2.960250	0.328500
232324	ADS1	CH1	2.960250	0.328500
234379	ADS1	CH0	2.957500	0.328625
234379	ADS1	CH1	2.957500	0.328625
236434	ADS1	CH0	2.959375	0.328625
236434	ADS1	CH1	2.959375	0.328625
215828	ADS2	CH0	2.958500	0.329125
215828	ADS2	CH1	2.958500	0.329125
217883	ADS2	CH0	2.958500	0.329250
217883	ADS2	CH1	2.958500	0.329250
226103	ADS2	CH0	2.959125	0.329125
226103	ADS2	CH1	2.959125	0.329125
228158	ADS2	CH0	2.959000	0.329000
228158	ADS2	CH1	2.959000	0.329000
230213	ADS2	CH0	2.959000	0.329000
230213	ADS2	CH1	2.959000	0.329000
232268	ADS2	CH0	2.959250	0.329000
232268	ADS2	CH1	2.959250	0.329000
234323	ADS2	CH0	2.958250	0.329125
234323	ADS2	CH1	2.958250	0.329125
236378	ADS2	CH0	2.958750	0.329000
236378	ADS2	CH1	2.958750	0.329000

Hình 10. Dữ liệu xuất trên TeraTerm

3.3. Thảo luận

Kết quả thực nghiệm cho thấy hệ thống hoạt động ổn định và đáp ứng tốt yêu cầu thu thập dữ liệu từ nhiều cảm biến. Dữ liệu từ hai module **ADS1115** được hiển thị đồng thời trên hai cổng **USART** mà không xuất hiện hiện tượng mất dữ liệu hoặc gián đoạn, chứng tỏ cơ chế xử lý song song đã được triển khai hiệu quả.

Đặc biệt, minh chứng rõ nét nhất cho khả năng hoạt động đa nhiệm song song được thể hiện qua các giá trị **Tick** (bộ đếm thời gian hệ thống của RT-Thread, thường 1 tick tương đương 1 mili-giây) được in ra trên phần mềm Tera Term. Quan sát dữ liệu thu được, khoảng thời gian hoàn thành một chu kỳ đọc của mỗi luồng là khoảng **2055** ticks (bao gồm 2000ms delay chủ động và khoảng 55ms cho việc giao tiếp phần cứng). Tuy nhiên, khi đối chiếu mốc

thời gian xuất dữ liệu giữa hai luồng tại cùng một thời điểm (ví dụ: **luồng 1 - COM3** ghi nhận Tick **226159**, trong khi **luồng 2 - COM6** ghi nhận Tick **226103**), độ lệch thời gian thực thi giữa hai tác vụ chỉ rơi vào khoảng hơn 50 ticks (tương đương 50 mili-giây).

Nếu hệ thống được lập trình theo phương pháp tuần tự truyền thống (blocking), luồng thứ hai sẽ buộc phải chờ luồng thứ nhất thực hiện xong toàn bộ quá trình bao gồm cả khoảng thời gian chờ 2 giây khiến độ lệch Tick giữa 2 công hiển thị phải lớn hơn 2000 ticks. Việc hai giá trị Tick bám sát nhau chứng tỏ rằng: ngay khi luồng 1 bắt đầu đi vào trạng thái "ngủ" (delay), hệ điều hành đã lập tức chuyển đổi và cấp quyền CPU cho luồng 2 hoạt động. Cả hai tác vụ thực chất đang song song thực hiện quá trình đếm lùi thời gian chờ

dưới nền, khẳng định sự ưu việt của cơ chế lập lịch trong RTOS.

Việc kết hợp giữa DMA và hệ điều hành thời gian thực RT-Thread đóng vai trò quan trọng trong việc nâng cao hiệu suất hệ thống. DMA cho phép truyền dữ liệu trực tiếp giữa ngoại vi và bộ nhớ mà không cần CPU can thiệp, trong khi RT-Thread đảm nhiệm việc điều phối các tác vụ theo cơ chế đa luồng. Nhờ đó, CPU không bị chiếm dụng trong các quá trình truyền dữ liệu lặp lại, giúp hệ thống hoạt động mượt mà và tận dụng tài nguyên hiệu quả hơn so với phương pháp lập trình tuần tự truyền thông.

Ngoài ra, việc tách riêng hai luồng xử lý cho hai module ADS1115 giúp hệ thống có thể thu thập dữ liệu đồng thời từ nhiều nguồn khác nhau. Điều này không chỉ cải thiện tốc độ thu thập mà còn tăng khả năng mở rộng khi cần bổ sung thêm cảm biến trong tương lai. Kết quả điện áp sau khi chuyển đổi từ dữ liệu thô cũng cho thấy quá trình xử lý và tính toán được thực hiện chính xác và ổn định theo thời gian.

Tuy nhiên, hệ thống vẫn tồn tại một số hạn chế nhất định. Tốc độ giao tiếp I2C tương đối thấp so với khả năng xử lý của vi điều khiển, do đó có thể trở thành điểm nghẽn khi số lượng cảm biến tăng lên. Bên cạnh đó, việc sử dụng nhiều luồng cũng đòi hỏi phải quản lý tài nguyên hệ thống một cách hợp lý để tránh xung đột hoặc giảm hiệu suất.

Trong tương lai, hệ thống có thể được cải tiến bằng cách sử dụng các giao tiếp tốc độ cao hơn như SPI hoặc tối ưu cấu hình I2C để nâng cao băng thông truyền dữ liệu. Ngoài ra, việc mở rộng số lượng module và tối ưu thuật toán xử lý cũng là hướng phát triển tiềm năng nhằm nâng cao hiệu năng và khả năng ứng dụng của hệ thống.

Tổng thể, giải pháp đề xuất đã đạt được mục tiêu đặt ra, chứng minh được hiệu quả của việc kết hợp DMA và RTOS trong việc tối ưu hóa hiệu suất CPU và nâng cao khả năng thu thập dữ liệu trong các hệ thống nhúng đa cảm biến.

4. KẾT LUẬN

Nghiên cứu đã thiết kế và triển khai thành công hệ thống thu thập dữ liệu song song sử dụng vi điều khiển STM32 kết hợp hai module ADC ADS1115. Bằng việc tích hợp cơ chế DMA và hệ điều hành thời gian thực

RT-Thread, hệ thống đã khắc phục được hạn chế của phương pháp xử lý tuần tự, đồng thời giảm tải đáng kể cho CPU.

Kết quả thực nghiệm cho thấy hệ thống hoạt động ổn định, dữ liệu được thu thập liên tục và đồng bộ từ nhiều cảm biến. Điều này chứng minh hiệu quả của việc áp dụng cơ chế xử lý song song trong các hệ thống nhúng.

Tuy vẫn còn một số hạn chế về tốc độ giao tiếp, giải pháp đề xuất đã đáp ứng tốt mục tiêu đề ra và có tiềm năng mở rộng trong các ứng dụng giám sát thời gian thực và hệ thống đa cảm biến.

TÀI LIỆU THAM KHẢO

- [1] Texas Instruments, "ADS111x Ultra-Small, Low-Power, I2C-Compatible, 16-Bit ADCs Datasheet," 2018.
- [2] STMicroelectronics, "AN2824 Application Note: Optimizing I2C communication using DMA in STM32 microcontrollers," 2018.
- [3] RT-Thread Development Team, "RT-Thread Programming Guide," 2020.